Лабораторная №5. Последовательностная логика

Устройства памяти и последовательностная логика

В цифровых схемах информация хранится и обрабатывается не только в зависимости от текущих входов, но и от **предыдущих состояний**. Такие схемы называются **последовательностными логическими устройствами**. В отличие от **комбинационных схем**, где выход зависит только от текущих входов, в последовательностной логике учитывается **внутреннее состояние системы**, которое может изменяться со временем.

Именно внутреннее состояние и позволяет реализовывать **память**, временно сохраняющую данные. Базовыми элементами такой памяти являются **триггеры**, а более сложные устройства — **регистры** и **счётчики**. Все они доступны в библиотеке Logisim под категорией «Память».

Триггеры как основа памяти

D-триггер		**T-триггер**		**JK-триггер**			**RS-триггер**		
(D)	**(Q)**	**(T)**	**(Q)**	**(J)**	**(K)**	**(Q)**	**(S)**	**(R)**	**(Q)**
0	0	0	Q	0	0	Q	0	0	Q
1	1	1	Q'	0	1	0	0	1	0
				1	0	1	1	0	1
				1	1	Q'	1	1	?

Триггер — это базовый элемент последовательностной логики, способный хранить **один бит информации**, то есть иметь два состояния: «0» или «1». Он представляет собой **последовательностное логическое устройство** с памятью, в котором новое состояние зависит от текущего входного сигнала и предыдущего состояния.

В англоязычной литературе триггер известен как **flip-flop**. Он имеет:

- Один или несколько **входов управления** (в зависимости от типа),
- Два выхода: **Q** (прямой) и **Q′** (инверсный).

Существует четыре основных типа триггеров:

- **D-триггер** (Data) — запоминает значение входа D при тактовом импульсе,
- **T-триггер** (Toggle) — инвертирует состояние при каждом импульсе, если T=1,
- **JK-триггер** (Jump-Kill) — универсальный триггер, может реализовать поведение всех остальных,
- **RS-триггер** (Reset-Set) — устанавливает или сбрасывает состояние.

Поведение каждого триггера задаётся **таблицей переходов** (или **таблицей истинности**), в которой указывается, каким будет состояние Q после тактового импульса в зависимости от текущего состояния и значений входов. Для изучения рекомендуется самостоятельно построить эти таблицы и реализовать схемы триггеров в Logisim.

Поведение различных типов триггеров можно описать словами следующим образом:

- **D-триггер** (триггер данных): при срабатывании тактового входа триггер принимает значение с входа D. Таким образом, на выходе устанавливается то же значение, что и на входе D в момент тактового импульса. Существует 4 типа D-триггера, срабатывающие по верхнему уровню, нижнему уровню, переднему фронту и заднему фронту.

- **T-триггер** (триггер переключения): при каждом срабатывании тактового входа триггер меняет своё состояние на противоположное.
- **JK-триггер**: при срабатывании тактового входа:
  - если оба входа J и K равны 0 — состояние не меняется;
  - если J = 1, K = 0 — устанавливается единица;
  - если J = 0, K = 1 — устанавливается ноль;
  - если оба входа J и K равны 1 — происходит инверсия текущего состояния.
- **RS-триггер** (установка-сброс): при срабатывании тактового входа:
  - если оба входа R и S равны 0 — состояние остаётся прежним;
  - если R = 1, S = 0 — состояние устанавливается в ноль (сброс);
  - если R = 0, S = 1 — состояние устанавливается в единицу (установка);
  - если оба входа R и S равны 1 — поведение считается неопределённым.

В цифровых схемах для хранения информации используются различные **устройства памяти**, к которым относятся триггеры, регистры и счётчики. Эти устройства относятся к классу **последовательностной логики**, поскольку их выход зависит не только от текущих входных значений, но и от **предыдущего состояния** — то есть они «помнят» свою историю. Это отличает их от комбинационных устройств, выход которых определяется исключительно текущими входами.

В рамках курса мы будем рассматривать **синхронные** устройства памяти, у которых изменения состояния происходят **только по фронту тактового сигнала**. В Logisim тактовый вход обозначается треугольником, а иногда маркируется символом **C** или словом **Clock**.

Триггер — базовое устройство памяти, способное хранить **один бит** информации. Существует несколько типов триггеров: D, T, JK и RS, названных по обозначениям их управляющих входов. Каждый триггер имеет два выхода — **Q (прямой)** и **Q' (инверсный)**. Значение на выходе Q отражает текущее состояние триггера, тогда как Q' — его логическое отрицание.

Триггеры могут быть **синхронными** и **асинхронными**. Синхронные срабатывают строго по тактовому импульсу, тогда как асинхронные могут реагировать на входы немедленно, без синхронизации. Некоторые триггеры имеют **разрешающий вход** (обычно обозначается **E**, от *Enable*). Однако наличие такого входа не делает триггер синхронным — это просто условие допуска изменения состояния.

В англоязычной литературе асинхронные триггеры часто называют **"прозрачными"** (*transparent*), так как при активном разрешающем входе входной сигнал напрямую влияет на выход. Синхронные же, наоборот, называются **"непрозрачными"** (*opaque*) — они обновляют выход только в строго определённый момент.

Важно помнить, что использование асинхронных устройств в схемах с обратными связями может привести к **возбуждению схемы** (неустойчивому состоянию). Чтобы этого избежать, необходимо хотя бы одно звено обратной связи реализовать через **синхронный триггер**.

На практике триггеры реализуются на логических элементах (а в конечном итоге — на транзисторах), и часто включаются с **обратными связями**, то есть выходы соединяются с входами.

Для построения полностью синхронного триггера часто применяют **двухступенчатую конструкцию** (*master-slave flip-flop*). Такая схема состоит из двух одинаковых триггеров, включённых последовательно: пока один открыт, другой закрыт, и наоборот. Это позволяет обновлять выход только при определённом фронте тактового сигнала.

Пример такого подхода — **двухступенчатый D-триггер**, в котором передняя ступень (master) принимает данные на переднем фронте, а задняя (slave) — обновляет выход на заднем фронте (рисунок 1).

.

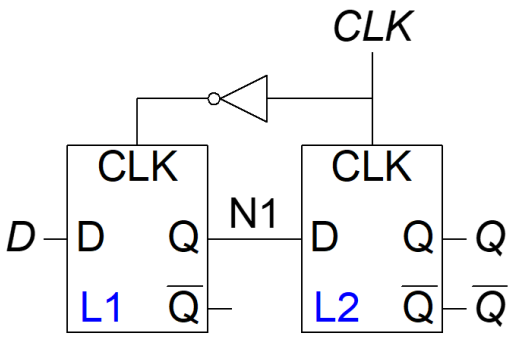


Рис. 1. Реализация синхронного D-триггера, работающего по переднему фронту.

Регистр — это устройство для хранения многобитных (n-разрядных) данных. В простейшем случае он состоит из группы D-триггеров, синхронно обновляемых одним тактовым сигналом. Регистр принимает данные на вход, сохраняет их при тактовом импульсе, и удерживает до следующей записи. В Logisim используется компонент **«Регистр»** с многобитными входом и выходом.

Счётчик — это регистр, который автоматически увеличивает (или уменьшает) хранимое значение при каждом тактовом импульсе. В Logisim это реализовано в виде компонента **«Счётчик»**, который можно настроить на инкремент, декремент, или работу как обычный регистр. Счётчик также имеет входы загрузки значения и выбора режима.

В Logisim также есть компонент **«Генератор случайных чисел»**, который при каждом тактовом импульсе выдает псевдослучайное значение заданной разрядности. Это удобно для тестирования работы схем на различных входах.

Если нужно сделать триггер, регистр или счётчик **асинхронным**, в его параметрах необходимо установить значение атрибута **«Срабатывание»** как «Высокий уровень». В этом случае устройство становится «прозрачным» и реагирует на уровень сигнала, а не на его изменение.

Более подробную информацию о поведении устройств памяти и их настройке в Logisim можно найти в документации библиотеки **«Память»**.

Задание 1. Сконструировать в Logisim **асинхронный RS-триггер** с использованием логических элементов (например, ИЛИ-НЕ). Проверить соответствие его поведения таблице истинности и сравнить работу схемы с аналогичным компонентом из встроенной библиотеки Logisim.

Задание 2. Реализовать **синхронный RS-триггер**, также на логических элементах. Убедиться, что поведение схемы корректно и соответствует спецификации.

Задание 3. Создать **синхронный D-триггер работающий по уровню**, используя базовые логические элементы. Поведение схемы должно совпадать с ожидаемым: при активном разрешающем сигнале входное значение должно передаваться на выход.

Задание 4. Смоделировать **синхронный D-триггер работающего по фронту**. Реализовать его можно по схеме двухступенчатого (master-slave) триггера. Убедитесь в корректной работе по тактовому импульсу. Сравнить работу с D-триггером из стандартной библиотеке Logisim.

Задание 5. На основе синхронного D-триггера из задания 3 реализовать **синхронный 8-разрядный регистр**. Все триггеры должны использовать общий разрешающий вход. Проверьте, что данные загружаются в регистр только при активном разрешении. Сравнить работу и с регистром из стандартной библиотеки Logisim.

Задание 6. Разработать **4-разрядный синхронный счётчик**, использующий триггеры из задания 4 и собственное **комбинационное устройство**, выполняющее следующую операцию: прибавляет 1 к текущему 4-битному входному значению; если вход

содержит максимальное значение (1111), на выходе должен быть ноль. Подключите выходы комбинационного устройства ко входам регистра, чтобы при каждом тактовом импульсе происходил инкремент. Сравните работу данного счетчика с работой счетчика из стандартной библиотеки Logisim.

Задание 7. Построить сумматор с возможностью запоминать промежуточный результат суммирования по схеме на рисунке 2.

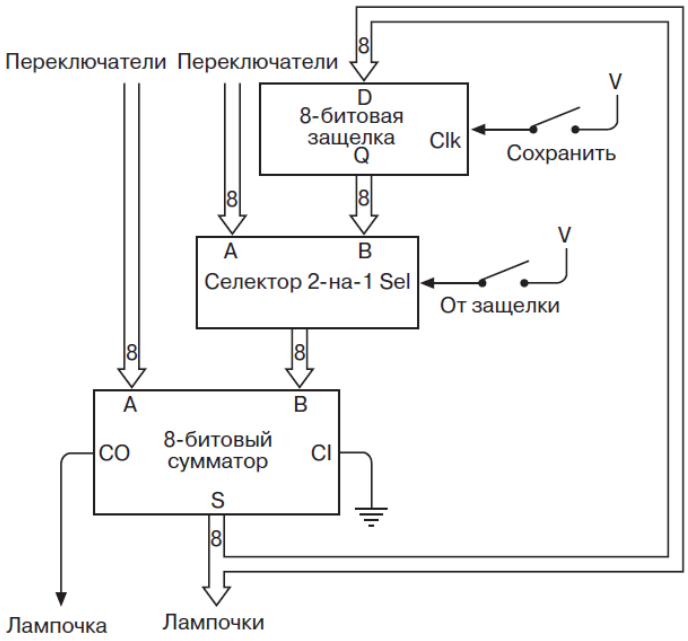


Рис. 2. Схема сумматора с памятью.

Задание 8. Построить сумматор с записыванием результата в аккумулятор по схеме на рисунке 3.

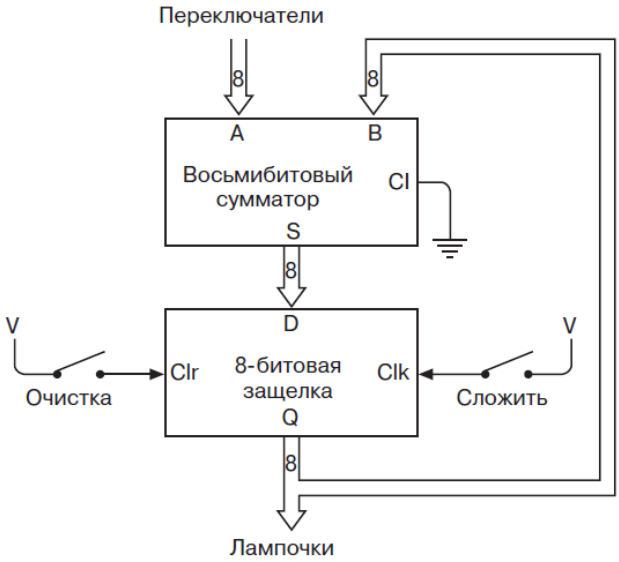


Рис. 3. Сумматор с аккумулятором.

Задание 9. Собрать память на 8 ячеек, каждая по 8 бит, используя плексоры и регистры из готовой библиотеке Logisim. Сравнить работу памяти с работой ОЗУ-памяти с раздельными портами чтения и записи из стандартной библиотеки Logisim.

Требования к выполнению работы

- Все задания необходимо реализовать **в одном проектном файле Logisim**.
- Каждое устройство (триггер, регистр, счётчик и т.д.) должно быть оформлено в **виде отдельной схемы** (*subcircuit*) с осмысленным и однозначным **названием схемы**, а также **всех её входов и выходов**.
- Все схемы и подсхемы должны быть аккуратно оформлены и проверены на корректную работу.

Дополнительные вопросы и задания (расширенный список)

Базовый уровень (проверка понимания)

1. Может ли существовать асинхронный D-триггер без разрешающего входа? Почему?
2. Чем отличается поведение D-триггера с разрешающим входом от поведения синхронного D-триггера?
3. Какую минимальную модификацию нужно внести в JK-триггер, чтобы получить T-триггер?
4. Почему на выходе RS-триггера возникает неопределённое состояние при одновременной подаче логической 1 на входы R и S?
5. Как работает инверсный (Q') выход триггера? Когда и зачем он может быть полезен?
6. Что произойдёт, если не задать начальное состояние триггера в Logisim? Как будет выглядеть его поведение при моделировании?

Средний уровень (проектирование и анализ)

7. Постройте временную диаграмму работы синхронного D-триггера при подаче произвольной последовательности сигналов на вход.
8. Какие преимущества и недостатки имеет двухступенчатая реализация синхронного триггера (Master-Slave)?
9. Можно ли использовать триггер в качестве логического элемента (например, ИЛИ)? Если да, то каким образом?
10. Предложите способ реализации **одного универсального триггера**, способного работать в режимах D, T, JK и RS, переключаемых специальным входом конфигурации.
11. Может ли счётчик работать без регистра? Чем счётчик отличается от регистра с точки зрения логики?
12. Как можно реализовать **модульный счётчик**, например, счётчик по модулю 10?

Продвинутый уровень (оптимизация и практическое применение)

13. Найдите способ минимизации количества логических элементов в синхронном регистре. Какие триггеры выбрать для этого?
14. В каких ситуациях использование прозрачных (асинхронных) триггеров предпочтительнее синхронных?
15. Как влияет задержка распространения сигнала на корректность работы схемы, содержащей несколько триггеров?

16. Попробуйте реализовать **сдвиговый регистр** и объясните, где он может быть применён.

17. Чем отличается реализация счётчика на основе сумматора от классической реализации на D-триггерах?

18. Реализуйте **линейный обратный сдвиговый регистр (LFSR)** и объясните, как он используется для генерации псевдослучайных последовательностей.

19. Как можно реализовать **двоично-десятичный счётчик (BCD)** на базе триггеров и логических элементов?

Исследовательские и нестандартные задания

20. Найдите и реализуйте альтернативный способ построения RS-триггера на элементах NAND, а не NOR. Объясните разницу в поведении.

21. Исследуйте, как изменение тактовой частоты влияет на стабильность работы регистров и счётчиков в Logisim.

22. Постройте схему, моделирующую простой **синхронный автомат (Mealy/Moore)** на основе триггеров и логических элементов.

23. Сравните поведение регистров при параллельной и последовательной подаче данных.

24. Разработайте **4-битный регистр с возможностью установки, сброса и хранения данных по управляющему слову**.

25. Реализуйте **регистр сдвига с циклическим сдвигом** (бит, сдвинутый вправо, возвращается влево).